



American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Course: Computer Graphics

**Project Title: VELOCITY RUN – 3D Endless Car Racing Game
using OpenGL**

Semester: Spring 25-26

Section: T

Group No. 6

Submitted to

MD SAMIUL ISLAM

Submitted by

<i>Group:6</i>		<i>Section:T</i>
SL	Student Name	Student ID
1	M. JAHAN WALID	24-56111-1
2	PRONoy RAHMAN	24-56088-1
3	ANKON SAHA	23-54172-3
4	ANANTA SAHA	23-54171-3
5	ANYKA AKTER AYNA	23-54139-3

Table of Contents

1	Introduction.....	3
2	Objectives	3
3	Tools and Technologies Used	3
4	System Overview	4
5	Project Features.....	5
6	Coordinate System and Graphics Concepts.....	5
7	List of Objects with Object ID	6
8	List of Functions	7
9	List of Animation Functions	8
10	Game Maps Description.....	8
11	User Controls	9
12	Implementation Details	10
12.1	Initialization	10
12.2	Display Function	10
12.3	Update System	10
12.4	Camera System	10
12.5	Terrain Rendering	10
12.6	Weather System.....	10
12.7	HUD System	10
13	Algorithms and Game Logic.....	10
13.1	Collision Detection Algorithm	10
13.2	Item Collection Algorithm	11
13.3	Endless Environment Algorithm.....	11
14	Lighting and Weather System	11
15	HUD and Scoring System	12
16	Output Description	13
17	Contribution Table	13
18	Challenges Faced	13
19	Future Improvements.....	13
20	Conclusion	14
21	References.....	14

1 Introduction

VELOCITY RUN is a 3D game of real time car race. It incorporates OpenGL and GLUT libraries for rendering interactive graphics. OpenGL is one of the most widely used graphic programming interfaces which can be used for both 2D and 3D graphics rendering and this game makes use of this along with the GLUT library. The topic falls under the domain of computer graphics.

It demonstrates various aspects of computer graphics such as rendering of 3D models, transformations, lighting, animations, interactions with the user, detecting collision, and the effect of the environment. The game is created as an endless runner game, in which the player has to drive a car and avoid obstacles while picking up objects along the way.

Some maps have diverse environments and play modes. The game employs sophisticated graphics such as fog effects, weather simulation, lighting transitions, terrain rendering, water effects, and head-up display (HUD) mechanisms for greater realism.

The main goal of this project is applying theoretical concepts of computer graphics into engaging gaming application using C++ and OpenGL.

2 Objectives

The aims of this assignment are:

- To study the basic aspects of OpenGL graphics programming.
- To develop a fully functional interactive 3D game environment.
- To use timer functions to animate in real time.
- To include translations and scaling in transformations.
- To add light effects and fog in 3D space.
- To develop collision detection and game physics.
- To design multiple maps with various appearances.
- To make the game interactive using keyboard control.
- To implement scoring, HUD, and game-over screens.

3 Tools and Technologies Used

Component	Technology
Programming Language	C++
Graphics Library	OpenGL
Utility Toolkit	GLUT

Component	Technology
Development Environment	CodeBlocks / Visual Studio
Platform	Windows
Rendering Type	Real-time 3D Rendering

4 System Overview

The program is a 3D infinite car racing game that contains a menu and four different maps. Each map features its own graphics, rendering, and game logic. This application features a modular structure; therefore, each map will retain its own environment and other elements.

Components of the system include:

- Main menu screen
- Four maps
- Player movement
- Obstacles which dynamically occur
- Collecting mechanism
- Collisions
- Weather elements
- Fog rendering
- Light
- Heads-up display
- Scoring
- Restart and game-over components

In the main game loop, the coordinates of objects are constantly updated, inputs from the user are processed, collisions are detected, and everything is redrawn with the help of OpenGL.



Figure 1 : Main Menu Interface of VELOCITY RUN

5 Project Features

The application involves a number of game and graphical features:

- Gameplay Features
- Continuous car race
- Obstacles that generate dynamically
- Coins and power-ups that can be picked up
- Shielding mechanism
- Speed booster
- Difficulty increases gradually
- Scoring
- Game-over and restart functionality
- Graphical Features
- Real-time 3D graphics
- Lighting
- Fog
- Terrains rendering
- Water simulation
- Weather effects (Rain and Snow)
- HUD
- Perspective camera
- Sun and night transitions
- Interaction Features
- Keyboard control
- Menu navigation
- Restarting ability
- Map choosing

6 Coordinate System and Graphics Concepts

A 3D coordinate system is employed to generate every object and scenery in the project.

Coordinate Axes:

- The X-axis: left-right direction
- The Y-axis: height
- The Z-axis: forwards-backwards distance

Depth sense is obtained using perspective projection by means of the `gluPerspective()` method.

Several important graphics concepts were used in this project:

- Translation
- Scaling
- Rotation
- Perspective projection
- Lighting
- Fog rendering
- Double buffering
- Real-time animation
- Texture-like color shading

Double buffering is used with `glutSwapBuffers()` to ensure smooth rendering and prevent screen flickering.

7 List of Objects with Object ID

Object ID	Object Name
Obj-01	Player Car
Obj-02	Road
Obj-03	Obstacle Car
Obj-04	Coin
Obj-05	Shield Power-up
Obj-06	Boost Item
Obj-07	Hills
Obj-08	Water
Obj-09	Trees
Obj-10	HUD
Obj-11	Fog Environment

Object ID	Object Name
Obj-12	Weather Particles
Obj-13	Sky Environment
Obj-14	Lighting Source

8 List of Functions

Function Name	Purpose
display()	Main rendering controller
update()	Main game update loop
keyboard()	Keyboard input handling
drawMenu()	Displays game menu
reset3D()	Resets 3D environment
map1_display()	Renders Map 1
map2_display()	Renders Map 2
map3_display()	Renders Map 3
map4_display()	Renders Map 4
map2_update()	Updates Map 2 gameplay
map2_drawHill()	Draws hill terrain
map2_drawWater()	Draws animated water
map4_drawHUD()	Draws HUD information

9 List of Animation Functions

Animation Function	Description
update()	Updates all animations
glutTimerFunc()	Timer-based animation loop
map2_update()	Controls movement and collisions
map2_drawWater()	Water wave animation
map2_particles[]	Weather particle system
map4_drawHUD()	Dynamic HUD updates

10 Game Maps Description

Map 1:

Map 1 illustrates a plain endless road with dynamic objects and challenging gameplay elements as the game progresses. It describes the fundamentals of the game.

Map 2:

Map 2 has improved visuals of the terrain, such as hills, dynamic water, fog, weather, and lighting effects. The emphasis of this map lies in aesthetics and immersive gameplay.

Map 3:

Map 3 creates a challenging environment through fast-moving obstacles and fast-paced gameplay.

Map 4:

Map 4 represents an advanced endless racing game with dynamic HUD, fast-paced gameplay, and extended features.

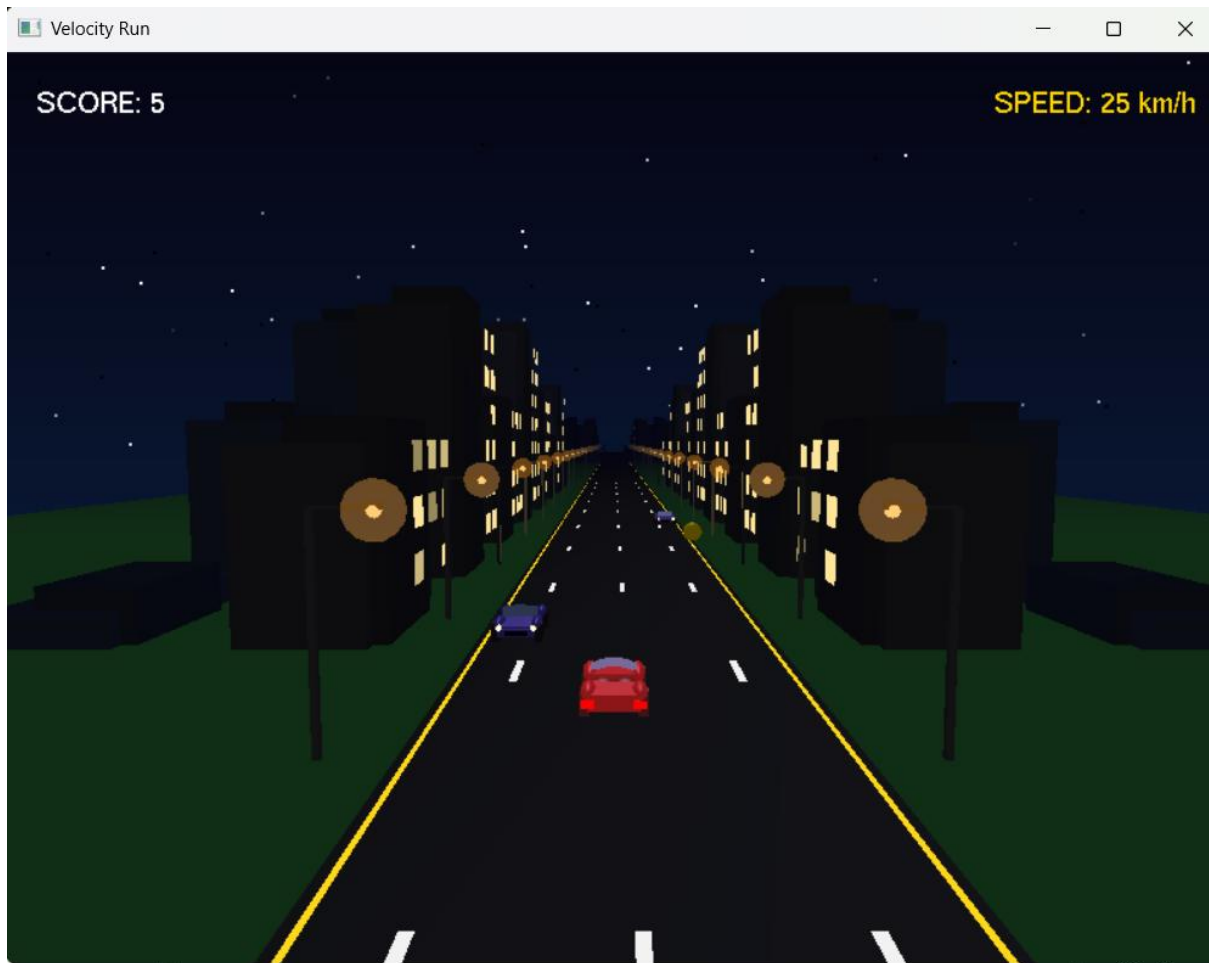


Figure 2: Gameplay with obstacles

11 User Controls

Key	Action
Left Arrow	Move Left
Right Arrow	Move Right
Up Arrow	Accelerate
Down Arrow	Brake
Space	Jump
R	Restart Game
ESC	Return to Menu

12 Implementation Details

12.1 Initialization

Initialize the OpenGL environment with a perspective view, depth buffer, and appropriate lighting and fog properties.

12.2 Display Function

The display system takes care of drawing all game objects and environments. There is a display function associated with each map that renders its world.

12.3 Update System

The update system ensures that object positions vary according to their movement, score, and collision detection is performed.

12.4 Camera System

A perspective camera helps in making the view more realistic and provides the experience of traveling ahead through the racing game environment.

12.5 Terrain Rendering

The hills in the terrain are generated dynamically by quad-based rendering and calculation of heights.

12.6 Weather System

Rain and snow effects are generated using particle generation systems that get updated all the time.

12.7 HUD System

The HUD displays important gameplay information including score, distance, speed, time, and player position.

13 Algorithms and Game Logic

13.1 Collision Detection Algorithm

- Refresh positions of the obstacles
- Compare positions of the obstacles with that of the player
- See whether there is any overlap between them on the x and z coordinates
- Initiate collision process if there is overlap
- Implement shield algorithm when the shield switch is active
- End game if collision occurs but the shield switch is off

13.2 Item Collection Algorithm

- Generate collectable items
- Find if there is any overlap between the player and a collected item
- Implement item power up effect
- Remove item from the playing area after collection

13.3 Endless Environment Algorithm

- Continue moving world objects towards the player
- Reset objects which have left the playing area
- Position newly created objects randomly into the scene
- Increase difficulty gradually with progression of the game

The collision detection system uses coordinate comparison and bounding-range checking to determine interactions between the player and obstacles.

The system supports:

- Obstacle collision
- Coin collection
- Shield activation
- Speed boost collection

Dynamic collision tolerance is implemented to ensure accurate detection at higher speeds.

14 Lighting and Weather System

Here are a few visualizations that have been used:

Lighting:

Using OpenGL lighting methods, environment lighting is simulated to enhance visibility.

Fog Effects:

Fog is implemented using OpenGL fog functions to increase environmental realism and depth perception.

Weather System:

The game includes:

- Rain
- Snow
- Changes in Environmental color
- Day and night blending

Environmental colors change dynamically to simulate atmospheric variations.

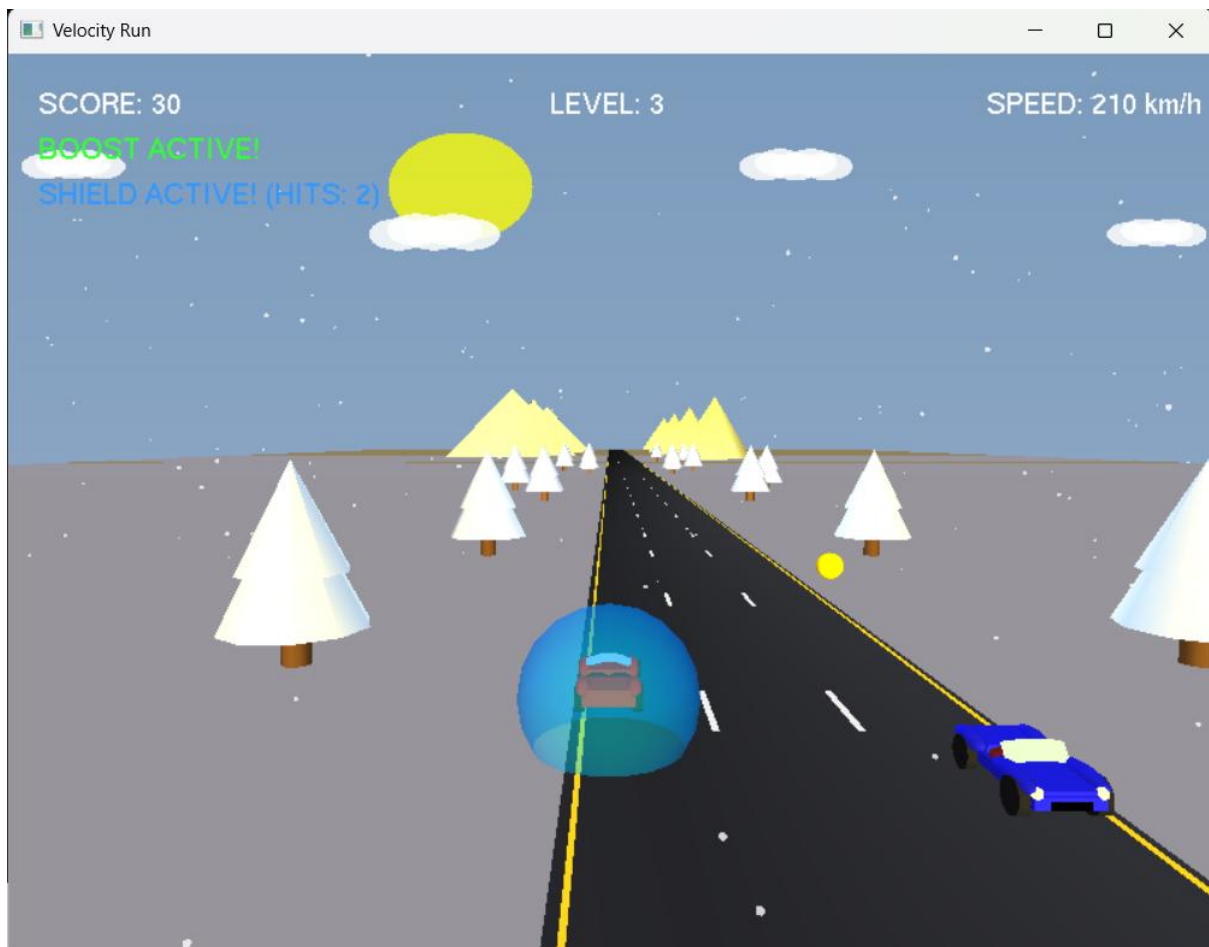


Figure 3: Snow Fall Weather Effects

15 HUD and Scoring System

The HUD system continuously displays gameplay statistics including:

- Score
- Distance traveled
- Current speed
- Game timer
- Player position
- Game-over messages
- Restart instructions
- Gameplay controls

16 Output Description

Output Window: It is fully featured in 3D racing games, including obstacles, terrain, lightings, weather conditions, and other gaming elements.

Menu Screen: The user gets options to choose any map out of four different maps available in the game.

Playing the Game: In the game, the player drives a car, avoids collision, and gains some objects to get scores.

17 Contribution Table

Member Name	Implemented Features	Contribution
M.JAHAN WALID	Menu system and HUD	20%
PRONoy RAHMAN	Collision and gameplay logic	20%
ANKON SAHA	Terrain and weather system	20%
ANANTA SAHA	Lighting and map design	20%
ANYKA AKTER AYNA	Structuring code and merge	20%

18 Challenges Faced

Several challenges were encountered during development:

- Creating smooth movement for the camera
- Implementing precise collision detection
- Improving speed of rendering terrain
- Handling both fog and lights simultaneously
- Ensuring stability during fast movement
- Creating multiple maps using different systems
- Avoiding issues while changing levels

These were some of the ways through which knowledge was gained regarding real-time graphics programming and optimization.

19 Future Improvements

Improvements that may be implemented in the future could be:

- Textures
- Sound and background music

- AI-driven cars
- Multi-player capabilities
- Physics engine
- Particle effects
- Realistic environment
- Mobile platform compatibility
- Additional maps and cars

20 Conclusion

VELOCITY RUN can be used as an example of a good way to apply computer graphics knowledge to practice using OpenGL and GLUT libraries. There are several important areas covered by the project, such as rendering 3D objects, transformations, animation, collision detection, lighting and fog effects, generating terrains, and interaction with users.

The project was helpful in gaining practical skills for developing a complex graphical game using C++ and OpenGL. In addition to maps and weather system, there is a heads-up display (HUD) and some other gameplay elements involved.

In general, the project is rather balanced between graphics and gameplay programming aspects.

21 References

1. OpenGL Programming Guide
2. GLUT Documentation
3. LearnOpenGL Official Tutorials
4. Computer Graphics Lecture Notes
5. OpenGL Official Documentation
6. Interactive Computer Graphics by Edward Angel